

RNN Character Prediction Application

```
import numpy as np

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import SimpleRNN, Dense

from tensorflow.keras.utils import to_categorical
```

```
def MarvellousCharcterPredictions():
```

```
    # 1) Prepare toy text dataset
```

```
    text = "hellohellohellohello" # repeating pattern
```

```
    chars = sorted(list(set(text))) # unique characters
```

```
    char_to_int = {c:i for i,c in enumerate(chars)}
```

```
    int_to_char = {i:c for i,c in enumerate(chars)}
```

```
    print("Unique characters:", chars)
```

```
    # 2) Convert text to integer sequence
```

```
    encoded = [char_to_int[c] for c in text]
```

```
    # 3) Create input-output pairs (sequence length = 3)
```

```
    seq_length = 3
```

```
    X, y = [], []
```

```
    for i in range(len(encoded) - seq_length):
```

```
        X.append(encoded[i:i+seq_length]) # e.g. "hel"
```

```
        y.append(encoded[i+seq_length]) # next char e.g. "l"
```

```
    X = np.array(X)
```

```
    y = np.array(y)
```

```
    # One-hot encode output
```

```
    y = to_categorical(y, num_classes=len(chars))
```

```
    # Reshape X for RNN (samples, timesteps, features)
```

```
    X = X.reshape((X.shape[0], X.shape[1], 1))
```

```

# 4) Build RNN model

model = Sequential()

model.add(SimpleRNN(16, activation="tanh", input_shape=(seq_length, 1)))

model.add(Dense(len(chars), activation="softmax"))


model.compile(loss="categorical_crossentropy", optimizer="adam", metrics=["accuracy"])


# 5) Train

model.fit(X, y, epochs=300, verbose=0)


# 6) Test prediction

test_input = ["h","e","l"] # we expect model to predict "l"

encoded_input = np.array([[char_to_int[c] for c in test_input]]).reshape((1, seq_length, 1))


pred = model.predict(encoded_input, verbose=0)

predicted_char = int_to_char[np.argmax(pred)]


print("Input:", test_input, "-> Predicted next char:", predicted_char)


def main():

    MarvellousCharcterPredictions()


if __name__ == "__main__":

    main()

```

Explanation of above application

- **Task:** Next-character prediction.
- **Granularity:** Character level.
- **Model:** A simple recurrent neural network (SimpleRNN) with one hidden layer (16 units) + a softmax output over the unique characters in the text.

Data preparation

Raw text and vocabulary

```
text = "hellohellohellohello"
chars = sorted(list(set(text))) # unique characters
char_to_int = {c:i for i,c in enumerate(chars)}
int_to_char = {i:c for i,c in enumerate(chars)}
```

- text length = **20** characters (“hello” × 4).
- Unique characters in “hello” = {h, e, l, o} → after sorted(...):
['e', 'h', 'l', 'o'].
- Example mapping (because of alphabetical sort):
 - e→0, h→1, l→2, o→3(Important: mapping depends on the **sorted order**, not the order of first

appearance.) **Integer encoding the text**

```
encoded = [char_to_int[c] for c in text]
```

- Converts each char to its integer id.
- This is **not** one-hot; it’s just integer indices (inputs will be reshaped to numeric features of size 1; Keras casts to float internally).

Sliding window to make (X, y) pairs

```
seq_length = 3
X, y = [], []
for i in range(len(encoded) - seq_length):
    X.append(encoded[i:i+seq_length]) # 3 chars as input
    y.append(encoded[i+seq_length]) # 4th char as label
```

- With length 20 and seq_length=3, you get **17** training samples (from positions 0..16).

- A few concrete examples from the actual text:
 - indices 0–2: **h e l** → next is **l**
 - indices 1–3: **e l l** → next is **o**
 - indices 2–4: **l l o** → next is **h**
 - indices 3–5: **l o h** → next is **e**
 - ...and so on (a repeating cycle from the “hello” pattern).

One-hot encode the targets

```
y = to_categorical(y, num_classes=len(chars))
```

- len(chars)=4, so y becomes shape **(17, 4)**.

- Reason: we are doing **multi-class classification** (which of the 4 characters comes next?), so the label must be a one-hot vector.

Reshape inputs for RNN

```
X = np.array(X) # (17, 3)
X = X.reshape((X.shape[0], X.shape[1], 1)) # (samples, timesteps,
features) → (17, 3, 1)
```

- **Timesteps = 3** (we feed 3 characters in order).
- **Features = 1** (each timestep is a single numeric feature: the integer id).
Note: In larger setups, we would typically use an **Embedding** or **one-hot inputs** instead of raw integers to avoid implying any “ordinal” meaning to ids.

Model architecture

```
model = Sequential()
model.add(SimpleRNN(16, activation="tanh", input_shape=(seq_length,
1)))
model.add(Dense(len(chars), activation="softmax"))
SimpleRNN(16, activation="tanh")
```

- **Recurrent layer** with 16 hidden units.
- Processes the sequence (3 timesteps) and returns the **last hidden state** (a vector of length 16).
- **Why tanh?** Values range in $[-1, 1]$, which helps stabilize recurrent dynamics. (ReLU is less common for vanilla RNNs due to exploding activations; LSTM/GRU are more robust alternatives.)

Parameter count (good to teach!):

- For SimpleRNN with **U** units and input dim **D**:
 - $\text{Weights} = U * (U + D + 1)$ (kernel $D \times U$, recurrent $U \times U$, bias U)
- Here **U=16, D=1** → $16 * (16 + 1 + 1) = 16 * 18 = 288$ parameters.

Dense(4, activation="softmax")

- Maps the 16-dim hidden state to 4 logits (one per character).
- **Softmax** converts logits to a probability distribution over the 4 characters.

Parameter count:

- Dense weights: $16 * 4 = 64$
- Bias: 4
- Total Dense params = **68**

Grand total = 288 (RNN) + 68 (Dense) = **356 parameters**.

Loss, optimizer, metrics

```
model.compile(loss="categorical_crossentropy", optimizer="adam",  
metrics=["accuracy"])
```

- **Loss:** `categorical_crossentropy` is the standard for one-hot multi-class classification. If **p** is the predicted probability vector and **y** is one-hot, the loss simplifies to $-\log(p_{\text{trueClass}})$.
- **Optimizer:** `adam` is a robust default that adapts learning rates per parameter.
- **Metric:** `accuracy` counts correct argmax predictions.

Training

```
model.fit(X, y, epochs=300, verbose=0)
```

- Trains on the 17 examples for **300 epochs**.
- With such a tiny dataset, the model will **overfit** (which is totally fine for a didactic demo—our goal is to make it memorize the pattern).
- No validation split is used here (again fine for a toy example).

Inference (predicting the next character)

```
test_input = ["h", "e", "l"] # we expect "l" next in the pattern  
encoded_input = np.array([[char_to_int[c] for c in  
test_input]]).reshape((1, seq_length, 1))
```

```
pred = model.predict(encoded_input, verbose=0) # shape (1, 4)  
predicted_char = int_to_char[np.argmax(pred)]
```

- `encoded_input` shape is **(1, 3, 1)** → one sample, 3 timesteps, 1 feature each. •

`pred` is a **(1, 4)** probability vector.

- `np.argmax(pred)` gives the predicted class id; map back via `int_to_char`. •

For “**h e l**”, the next character in the repeating pattern is indeed “**l**”.

